

Barkeep: SCF Instaward application

A spending-capped, payee-limited agent account on Stellar.

Applicant: Barbaros Emre Alagöz · Stellar Türkiye Chapter · Prepared 2026-09-25

Status:

- **Stellar Testnet only.**
- **Unaudited contracts.**
- **No external users yet.**

Every claim below links to its proof. My own activity is labelled **[TEAM · TESTNET]** or **[TEAM · ARC MAINNET]**. It is kept separate from **[EXTERNAL]** usage, of which there is none yet.

Fields marked **[CHAPTER: ...]** are for the Chapter Lead to confirm. SDF does not publish the Instaward form, so the sections below follow what the [Instawards Official Rules](#) ask a submission to contain. They will be moved into the form's own order once the Chapter Lead shares it.

Applicant and Chapter Lead

Field	Value
Applicant	Barbaros Emre Alagöz, a single-person team. GitHub: barbarosalagoz
Chapter	Stellar Türkiye Chapter (chapter page)
Chapter Lead assisting this submission	[CHAPTER: name of the Chapter Lead]
Chapter participation	[CHAPTER: participation as the Chapter Lead assesses it (Rules §4)]
Requested amount	[CHAPTER: the Instaward tier, within the \$1,000–\$5,000 initial range (Rules §6)]
Repository	github.com/barbarosalagoz/barkeep , Apache-2.0

Summary

What exists. A human opens a *tab* for an AI agent. A tab is a context rule on an OpenZeppelin smart account, with:

- a cap;
- a list of payees;
- an expiry;
- a session key made for that tab only.

The agent pays for HTTP requests over x402 against the tab. The cap and the payee list are enforced on chain, not by the agent or its server. All of this runs on Stellar Testnet.

The 30-day sprint adds three things:

- a starter kit that sets this up with one command;
 - a guide in English and Turkish;
 - one Türkiye chapter workshop, where developers make real Testnet transactions with their own keys.
-

In my own words

1. Why agents need a hard spending cap

Right now, agents spend tokens without any real control, and we can end up with nothing to show for the tokens that went out. That is why we put a hard spending cap in place and move forward with a short, focused build, the way a senior engineer would, and start from the result. When I drew up the project's first architecture, this was already obvious.

2. Why the cap lives on chain

Keeping the cap on chain means the limit is enforced by the same account that holds the money. A rule on a server can set a limit too, but if that server or the agent's key is compromised, the rule can be bypassed. On chain, even a stolen agent key can only spend up to the tab's cap, only to its payees and only before expiry. I proved this on Arc mainnet with Barkeep Arc, where every refusal case has an on-chain revert hash.

Evidence.

[TEAM · ARC MAINNET], reverted on chain:

- over the per-call maximum: [0x332409fb...](#)
- payee not on the list: [0xe2a02e84...](#)
- after expiry: [0x3d966f20...](#)

Source: [MAINNET.md](#).

[TEAM · TESTNET] On Stellar, refused on chain:

- over the cap: #3221, [6f001e4a...](#)
- payee not on the list: #3901, [681ae9ea...](#)
- after expiry: #3002, [b253a6f8...](#)

3. Why a narrow scope, and why a workshop

This is a very wide field, and the structure can easily grow if needed. That is exactly why it matters that developers focus on a single area. As a trainer, my goal is for developers to focus on one problem and one architecture, strengthen how they use it, and then extend the structure themselves.

4. Why me

I was the trainer for Rise In's first Stellar developer cohort, and we took the first steps of Stellar developer education at Rise In.

Evidence. [CHAPTER: confirmation from Rise In, or a link to the cohort]

5. What success looks like after 30 days

After 30 days, at least 10 developers from the workshop have made real transactions with their own keys, recorded separately from my own activity. The improvements Stellar provides and recommends are defined quickly, added to the architecture and turned into an ongoing process, so that it can scale and keep going.

Sections 1–5 are written from Barbaros's own notes in Turkish; English translation and light editing assisted by AI. One factual correction made with AI assistance, approved by Barbaros.

The sprint deliverable

There are three parts, all on Stellar Testnet:

1. **A starter kit.** It is a template repo. One command gives a builder:
 - an OpenZeppelin smart account;
 - a tab: a context rule with a session key made for that tab only, a spending-limit policy, a payee allowlist and an expiry;
 - a local x402 seller;
 - one paid request.
2. **A role-based guide in English and Turkish.** It has one section each for the agent builder, the seller and the human signer.
3. **One Türkiye chapter workshop.** Developers open a tab and make a payment with keys they generate themselves.

No new contracts are written. The contracts already exist, are tested, and are deployed on Testnet ([deployments/testnet.json](#)).

Relevance to Stellar

- It is built on OpenZeppelin `stellar-accounts` context rules and the `spending_limit` policy. The account's `__check_auth` enforces them on chain ([HOW_IT_WORKS.md](#)).
- It pays over x402 `exact` on Stellar with `@x402/stellar` ([ARCHITECTURE-v2.md §3.1](#)).
- It adds a payee-allowlist policy, which `stellar-accounts` does not ship ([contracts/barkeep-payee-allowlist](#)).
- The friction it documents is already reported upstream (see [Upstream work](#)).

Why not the existing path

The existing path is OpenZeppelin's smart-account docs plus the x402 quickstart. It never shows a builder how to give an agent a key that is capped and payee-limited on chain. A builder who tries it with a smart account fails first at the x402 client ([x402#3158](#)), then at the facilitator ([x402#3352](#), [x402#3515](#)), and is not told why.

How it composes with x402 and OpenZeppelin smart accounts

Layer	What Barkeep uses	Its own code	Status
Account	OpenZeppelin <code>stellar-accounts</code> =0.7.2: context rules, <code>spending_limit</code>	A thin account contract and its verifiers	On Testnet
Payee limit	The <code>stellar-accounts</code> policy interface	<code>barkeep-payee-allowlist</code>	On Testnet, 19 unit tests
Agent key	One ed25519 session key per tab, checked by the verifier contract	agentKeys.ts	Offline-tested. The Testnet done-test is the sprint's first task
Human signer	WebAuthn passkey through a verifier contract	<code>barkeep-verifier-webauthn</code>	Passkey-signed transfer: f6a66a60...

Layer	What Barkeep uses	Its own code	Status
Payment	x402 v2 exact , @x402/stellar 2.25.0	A client scheme that signs as the smart account (x402Scheme.ts), plus a facilitator with two checks relaxed (facilitator.ts)	Works only with a facilitator that accepts smart-account payers
Agent interface	MCP (stdio)	<code>open_tab</code> , <code>pay_and_fetch</code> , <code>tab_status</code> , <code>close_tab</code>	Testnet

The facilitator fork is a workaround, not the goal. Each relaxed check is commented with the upstream check it replaces. The kit will move to the stock facilitator once upstream accepts smart-account payers.

Not composed today:

- MPP (@stellar/mpp), deferred in [ARCHITECTURE-v2.md §3.2](#);
- the OpenZeppelin Relayer x402 facilitator, which has not been measured with a smart-account payer.

What exists today

[EXTERNAL] Independent third-party usage

None. There are no external users, partners, revenue or third-party deployments. The table that will record them is [EXTERNAL_USAGE.md](#), and it is empty.

[TEAM · TESTNET] Barkeep on Stellar Testnet

All of this was made by me, with my own keys. Every hash is in [deployments/testnet.json](#).

Behaviour	Result	Evidence
Transfer under the cap	Succeeds	15333d0a...
Transfer over the cap	Refused, #3221 SpendingLimitExceeded	6f001e4a...
Transfer after expiry	Refused, #3002	b253a6f8...
Transfer after <code>close_tab</code>	Refused, #3000 ContextRuleNotFound	afda2b58...
x402 payment through <code>pay_and_fetch</code>	Paid 0.0001 TAB	18baa88e...
x402 payment over the remaining cap, forced on chain	Refused, #3221	a5fc48c6...
Payee on the allowlist	Paid	42c1ad39...
Payee not on the allowlist	Refused, #3901 PayeeNotAllowed	681ae9ea...
Agent tries to add a payee	Refused, #3002	ff51d3a1...
Empty payee list forced into a rule	Refused, #3902 EmptyAllowlist	b195c7ff...
Full demo through the MCP server, 2026-09-14	3 payments, then close	7f0ec6c1... , a63f510b...
Public facilitator against a capped smart account	Refused (fee ceiling, event check). Settles with both checks relaxed	c02f9035... , finding 01

These runs used the shared agent key from before per-tab keys existed.

Tests on 2026-09-25:

- `cargo test --workspace`: 70 passing, 6 ignored;
- offline unit tests: 143 passing.

CI runs both, plus lint, typecheck and build, on every PR ([ci.yml](#)).

[TEAM · ARC MAINNET] The same design on Circle's Arc mainnet

This is a different chain. It is included to show the design ran with real funds. All of it was made by me.

Behaviour	Evidence
3 × 0.001 USDC x402 payments, settled by Circle's hosted facilitator	0x2b26d71f... , 0xfcc39bc0... , 0x210a8dfb...
Refusals: over per-call maximum, payee not listed, after expiry	Reverts in section 2 above
Close; the balance swept back	0xa0be0731...

Source: [barkeep-arc MAINNET.md](#).

Upstream work

All of these are still open. No maintainer has replied as of 2026-09-25.

Item	My role
OpenZeppelin/stellar-contracts#897	Author: confirmed the v0.9.0 smart-account client flow on Testnet
x402#3352	Comment with Testnet measurements supporting fix PR #3399
x402#3515	Author: the facilitator's fee-ceiling default refuses smart-account payers
x402#3158	Worked around in Barkeep's client scheme. The fix PR #3018 is by another builder

Success criteria and acceptance tests

Each criterion can be checked at the end of the sprint by someone other than me.

#	Criterion	Acceptance test	Evidence produced
S1	The kit opens a capped, payee-limited tab on Testnet from a clean machine	Someone other than me follows the README on a fresh machine, and has a tab open plus one paid request within 30 minutes. The run is timed and recorded	Tx hashes, a screen recording
S2	The kit is checked	A CI job builds the kit and runs its offline tests on every PR. The per-tab key done-test (pay, close, refused after close) is run on Testnet and its hashes recorded	CI run link, tx hashes
S3	Guide in English and Turkish	Both merged. Every command in them has been run, with its output recorded	Commit link
S4	Workshop held through the Türkiye chapter	Event page or chapter announcement	Link
S5	At least 10 developers from the workshop make real Testnet transactions with their own keys	Each hash is listed, with that person's written consent, in EXTERNAL_USAGE.md , separate from my own activity	Tx hashes, consent record
S6	Feedback	A short anonymous form. Results are published unedited	Link

Timeline (30 days)

Day 1 is the approval date. The dates assume approval on Monday 2026-10-12 and move with it.

Days	Dates	Work	Exit check
1-7	10-12 → 10-18	Extract the kit from <code>packages/mcp-server</code> as one command. Run the per-tab key Testnet done-test. Add the CI job	S2
8-12	10-19 → 10-23	English guide, by role. A clean-machine dry run by someone else	S1
13-16	10-24 → 10-27	Turkish guide, workshop material, consent form, feedback form	S3
17-21	10-28 → 11-01	Workshop, on [CHAPTER: date and venue]	S4
22-27	11-02 → 11-07	Fix what the workshop exposed. Collect consented hashes	S5
28-30	11-08 → 11-10	Sprint report: each criterion met or not, evidence, feedback	S6

Budget

The team is one person. Everything runs on Testnet, so the on-chain cost is \$0.

Line	Person	Hours
Kit: one-command setup, per-tab key done-test, CI job	Barbaros	30
English guide, by role	Barbaros	16
Turkish guide	Barbaros	8
Workshop material and dry run	Barbaros	10
Workshop delivery and on-site support	Barbaros	6
Consent and usage collection, external-usage table	Barbaros	6
Fixes after the workshop	Barbaros	10
Sprint report	Barbaros	6
Total		92

- **Venue:** [CHAPTER: provided by the chapter?]
- **Amount:** [CHAPTER: tier]. At the top of the initial range, \$5,000 for 92 hours is about \$54 per hour.

Risks and single points of failure

- **One-person team.** If I stop, the project stops.
 - There is no second maintainer and no second key-holder.
 - The work is in a public Apache-2.0 repo, and every deployment can be reproduced ([DEPLOYMENTS.md](#)).
- **Unaudited contracts.** None of Barkeep's contracts is audited.
 - OpenZeppelin's own audits cover the `stellar-accounts` library at v0.7.0-rc.1, four tags behind the 0.7.2 that Barkeep uses ([ARCHITECTURE-v2.md §4.1](#)).
 - The sprint stays on Testnet.
- **A known gap in the deployed account.** Under 0.7.2, the auth digest is not scoped to the account.

- A cross-account replay was accepted on Testnet ([512dbcbf...](#)) and refused on v0.9.0 ([dbeb6f62...](#)).
- The fix is to migrate when 0.9.0 is published.
- **Upstream dependency.** Stock x402 does not accept smart-account payers yet.
 - The workshop uses Barkeep's facilitator fork, and says so.
 - Upstream merges are not a success criterion.
- **Keys.**
 - **Long-lived Testnet keys** are generated and stored in the Stellar CLI key store, outside the repo. They are never rotated. They are throwaway Testnet accounts.
 - **Agent keys** are made per tab and destroyed on close.
 - **The MCP server still holds the admin key.**
 - **Arc mainnet:** the owner key was generated fresh for each funding attempt. Unused keys were destroyed. Agent keys were destroyed on close. On 2026-09-24 the owner and seller keys were shredded, and a full disk search found no copies.
 - **An Arc Testnet machine was wiped on 2026-09-21 with no backup,** and its Testnet keys were lost. That is disclosed in full.

Full record: [KEY_MANAGEMENT.md](#).

Competitive landscape

Sources were read on 2026-09-25.

Product	Where the limit is enforced	Payee limit	Stellar
Circle Agent Wallets	Off chain, in Circle's MPC	Recipient allowlists	Not listed
Coinbase CDP Policy Engine	Off chain, in a TEE	Address rules	No
Coinbase Spend Permissions	On chain, EVM	No	No
Safe Allowance Module , Zodiac Roles	On chain, EVM	Roles: through conditions	No
MetaMask ERC-7715/7710 , ZeroDev , Biconomy	On chain, EVM	Through call policies	No
Crossmint Agent Wallets	Claimed on chain; agent key in a TEE	Claimed	Listed; mechanism not verified
Privy , Turnkey	Off chain, in an enclave	Yes	Not verified
Stripe Shared Payment Tokens , Visa, Mastercard, AP2	Card network or protocol mandates	Per seller or mandate	No

On Stellar:

- OpenZeppelin `stellar-accounts` ships `spending_limit` but no payee policy. Barkeep builds on it.
- SDF's x402 roadmap names an x402-MCP server with smart-wallet spending policies. That is the closest overlap. It is on the roadmap only, as far as checked ([stellar.org](#)).
- The SCF #45 facilitator projects are facilitators, not payer-side policies.

What this means for Barkeep. Payee-limited on-chain caps already exist on EVM, so Barkeep is not a new idea in general. What does not exist yet is this: on Stellar, with OpenZeppelin accounts, paying over x402, the pieces do not fit together today. Barkeep documents exactly where they break, and works around it.

AI-assisted development

Sections 1–5 are written from Barbaros's own notes in Turkish; English translation and light editing assisted by AI. One factual correction made with AI assistance, approved by Barbaros.

- **The rest of this application** was drafted with Claude Code from the repository record.
- **The code:** most of the repository was written with Claude Code. 47 of 62 commits on `main` carry a Claude co-author trailer, including every commit touching the contracts and the MCP server.
- **Quality checks:** CI runs the tests on every PR. On-chain claims come from recorded Testnet runs, with refusals forced on chain.
- **Not in place:** branch protection, mutation testing and external review.
- **Findings 03 and 06:** Found by Barbaros.

Full statement: [AI_ASSISTED_DEVELOPMENT.md](#).

After the sprint

- There is no revenue model today, and this application does not claim one.
- If the sprint succeeds, the next step is a follow-on Instaward. That scope would be upstream compatibility:
 - testing and supporting the x402 fixes for smart-account payers ([#3018](#), [#3399](#), [#3515](#));
 - offering the payee-allowlist policy to OpenZeppelin `stellar-accounts`.
- A larger grant would only be sought after S5 shows real external usage.